Git y GitHub + Docker

Guía completa de referencia para programadores · Agenda de contactos · Documento didáctico

Este PDF cubre dos herramientas que todo programador debe dominar: **Git y GitHub** (control de versiones, historial, ramas, pipelines) y **Docker** (empaquetado y ejecución de aplicaciones en contenedores). Está pensado como referencia: léelo una vez y vuelve a las secciones que necesites.

Parte I — Git y GitHub: qué son, comandos esenciales, ramas, merge vs rebase, deshacer cambios, buscar commits, borrar archivos del historial y pipelines con GitHub Actions.

Parte II — Docker: contenedores vs máquinas virtuales, imágenes, Dockerfile, contenedores, volúmenes, redes, docker-compose y buenas prácticas.

PARTE I: Git y GitHub

1. Qué es Git y qué es GitHub

Git es un sistema de control de versiones **local**: un programa instalado en tu PC que guarda cada versión de tus archivos en una carpeta oculta (`.git`). Funciona sin internet.

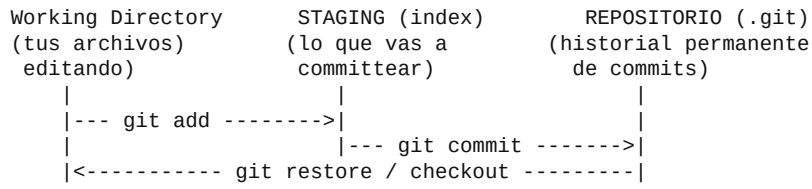
GitHub es un sitio web (empresa Microsoft) que aloja repositorios Git en la nube y añade funciones sociales: pull requests, issues, pipelines (Actions), revisión de código, páginas web, etc. Alternativas: GitLab, Bitbucket.

> **Idea clave:** Git es el motor; GitHub es el garaje en la nube donde guardas el auto y donde colaboran otros mecánicos. Puedes usar Git perfectamente sin GitHub.

2. El modelo de datos: cómo guarda Git las cosas

Git no guarda "diferencias": en cada commit guarda una **fotografía completa** del proyecto, identificada por un hash único de 40 caracteres (SHA-1), p. ej. `45f668c9a1b...`.

Las zonas de trabajo son el corazón de todo:



- **Working directory**: los archivos tal como los ves y editas.
- **Staging / índice**: lista de cambios que preparaste con `git add` para incluirlos en el próximo commit.
- **Repositorio**: historial de commits; cada commit apunta a su commit padre (forma una lista enlazada).
- **HEAD**: puntero al commit en el que estás parado (normalmente a la rama actual).

3. Comandos esenciales del día a día

```
git init                          # crea un repositorio en la carpeta actual
git clone URL                     # copia un repo remoto a tu PC
git status                        # qué hay modificado / staged / sin trackear
git add archivo.py                # pone UN archivo en staging
git add .                         # pone TODO lo nuevo/modificado en staging
git commit -m "mensaje"           # crea un commit con lo que había en staging
git log --oneline                 # historial resumido (hash + mensaje)
git log --oneline --graph --all   # historial con ramas y gráfico
git diff                          # cambios sin stagear (working vs staging)
git diff --staged                 # cambios ya staghead (staging vs último commit)
git show HASH                     # qué cambió en ese commit exacto
```

Mensajes de commit: escribe por qué cambiaste, no qué cambiaste (el diff ya lo dice). Convención popular: imperativo y corto, p. ej. `Agrega messagebox de confirmación al borrar`. Conviene un commit = una idea; no metas 10 cambios distintos en uno solo.

4. Ramas (branches)

Una rama es solo un **puntero móvil** a un commit. Crear ramas es baratísimo: no duplica archivos.

```
git branch login-rama             # crea la rama sin cambiarte a ella
git switch login-rama             # te cambia a ella (crea si no existe con -c)
```

```
git switch -c login-rama          # crea Y te cambia en un solo comando
git switch main                   # vuelve a main
git branch                        # lista tus ramas locales
git branch -d login-rama          # borra la rama YA fusionada (seguro)
git branch -D login-rama          # la borra aunque no esté fusionada (a la fuerza)
```

Rama principal: convención moderna `main` (antes se llamaba `master`). En tu proyecto la rama actual es `agenda-contactos`. Flujo típico: una rama por funcionalidad o corrección, se fusiona a main y se borra.

5. Merge: fusionar ramas

Merge = combinar el historial de otra rama en la actual. Hay 2 casos:

- **Fast-forward (avance rápido)**: si main no avanzó desde que creaste la rama, Git solo mueve el puntero main al último commit de la rama. No crea commit nuevo ni línea de historial extra.
- **Merge real (three-way)**: si ambas ramas avanzaron, Git crea un **commit de merge** con dos padres que une las dos líneas de historial.

```
git switch main
git merge login-rama
```

Conflictos

Si ambas ramas modificaron **las mismas líneas** del mismo archivo, Git no adivina qué quedarse y deja el conflicto marcado en el archivo:

```
<<<<<<< HEAD
messagebox.showinfo("Información", "Guardado")
=======
messagebox.showinfo("Listo", "Guardado")
>>>>>>> login-rama
```

Editas el archivo (borras los marcadores y dejas lo que debe quedar), luego:

```
git add archivo.py                 # marcas el conflicto como resuelto
git commit                        # cierra el merge
# o para abortar todo y volver como estaba:
git merge --abort
```

6. Merge vs Rebase: la diferencia clave

Ambos combinan código de otra rama, pero **reescriben el historial de formas distintas**.

merge: conserva TODO tal cual pasó

```
      A---B---C  main
     /         \
D---E---F---G   login-rama
               (merge crea H con dos padres)
git switch main
git merge login-rama
# resultado:     A---B---C---H  main
#               /         /
#          D---E---F---G
```

rebase: "rebobina" tus commits y los vuelve a poner encima

```
      A---B---C  main
     /         \
D---E---F---G   login-rama

git switch login-rama
git rebase main
# resultado:             A---B---C  main (HEAD)
                              \
                               D'--E'--F'--G'   login-rama
# D..G se REESCRIBEN: son copias nuevas (hash nuevos)
```

Tabla comparativa

```
| Criterio        | merge                       | rebase                       |
|-----------------|-----------------------------|------------------------------|
| Historial       | lo conserva tal cual        | lo reescribe (hash nuevos)   |
| Linealidad      | puede crear "cruces"        | historial 100% lineal        |
| Peligro         | ninguno                     | peligroso en ramas compartidas |
| Uso típico      | fusionar rama terminada     | actualizar TU rama con main  |
```

¿Cuándo usar cada uno?

- Para **traer los cambios de main a tu rama** y mantenerte al día: `rebase` (historial limpio).
- Para **fusionar tu rama terminada en main**: `merge` (deja constancia de que existieron las dos líneas).

> **Regla de oro:** NUNCA hagas rebase de una rama que ya subiste a GitHub y que otra persona usa. Reescribir historial compartido rompe el trabajo de los demás (para "arreglarlo" habría que hacer force push, que también es peligroso). Rebase solo en ramas locales o personales.

Rebase interactivo (el más útil)

Sirve para reordenar, unir, editar o borrar TUS commits antes de publicarlos, para que parezca que trabajaste "de corrido":

```
git rebase -i HEAD~3              # reescribe los últimos 3 commits
# se abre un editor con una línea por commit:
pick  a1b2c3 Agrega .gitignore
squash d4e5f6 corrige typo        # une este en el anterior
reword 9a8b7c Mejora mensaje      # solo cambia el mensaje
drop  1c2d3f WIP basura           # elimina ese commit
# guardas y Git reconstruye los commits
```

7. Trabajar con el remoto: fetch, pull, push

Un **remoto** es la copia del repo en la nube. Por defecto se llama `origin`.

```
git remote -v                      # ver la URL del remoto
git remote add origin URL          # conectar un repo de GitHub
git push -u origin main            # sube main y la vincula con upstream
git push                           # después de lo anterior, basta "git push"
git push origin main:feature-x     # sube tu main como rama feature-x
```

fetch vs pull (muy preguntado)

```
git fetch origin
# solo DESCARGA los nuevos commits del remoto a refs/remotes/origin/*
# NO toca tus archivos ni tus ramas. Es seguro: mirar sin tocar.
# Ideal para revisar: git log HEAD..origin/main

git pull origin main
# = git fetch origin  +  git merge origin/main
# descarga Y fusiona (o rebasea con --rebase) en tu rama actual
```

- **fetch**: actualizar el radar. No rompe nada. Úsalo cuando quieras ver qué hay nuevo.
- **pull**: actualizar y aplicar a tu rama. Puede generar conflictos ahí mismo.

Cuando `git push` te rechaza con *"non-fast-forward"* significa que el remoto tiene commits que tú no tienes: haz `git pull`, resuelve conflictos si los hay, y vuelve a push.

8. Deshacer: reset, revert, restore, stash

8a) Cambios sin commit: git restore

```
git restore archivo.py             # descarta cambios en el archivo (working)
git restore --staged archivo.py    # lo saca del staging, lo deja modificado
```

8b) Borrar un commit ya hecho: reset (rama LOCAL)

```
git reset --soft HEAD~1            # deshace el commit PERO deja todo en staging
git reset HEAD~1                   # deshace commit y staging (--mixed, el default)
git reset --hard HEAD~1            # deshace commit, staging Y archivos: TODO se borra
```

--hard es destructivo: los cambios no commiteados se pierden para siempre. Antes de usarlo, pasa por `git stash` o `git status` para no lamentarte.

8c) Deshacer un commit en una rama PUBLICADA: revert

```
git log --oneline                  # encuentra el hash del commit malo
git revert HASH                    # crea un commit NUEVO que "deshace" los cambios
git push                           # seguro: no reescribe historial
```

reset vs revert: reset **borra** el commit del historial (solo seguro si nadie más lo tiene); revert **agrega un commit que anula** los cambios (seguro siempre, es la forma profesional de deshacer en ramas compartidas).

8d) Guardar cambios a un lado: git stash

```
git stash push -m "wip formulario"  # guarda cambios temporales y limpia el working
git switch main                     # ahora puedes trabajar en otra cosa
git stash list                      # ve tus stashes guardados
git stash pop                       # recupera el último stash y lo borra de la lista
git stash apply                     # recupera el último stash sin quitarlo de la lista
```

9. Cómo encontrar commits: log, blame, bisect

```
git log --oneline -10               # últimos 10 commits
git log --oneline --since="2 weeks ago"
git log --oneline -- archivo.py     # solo tocaron ese archivo
git log -p archivo.py               # con el diff completo
git log --grep="login" -i           # busca por mensaje de commit
git blame archivo.py                # QUIÉN escribió cada línea y en qué commit
git bisect start                    # busca BINARIA el commit que rompió algo
git bisect start HEAD bad           # commit actual = malo
git bisect good 45f668c             # commit viejo = bueno
# Git te lleva al commit del medio, pruebas, y dices:
git bisect good                     # o  git bisect bad
# en pocos pasos encuentra el culpable exacto
git bisect reset                    # termina el modo bisect
```

Para "regresar a un commit en específico": `git switch --detach HASH` te deja en modo lectura para inspeccionar; `git reset --hard HASH` mueve la rama actual allá (destruye lo posterior); `git revert HASH` si el commit ya está publicado. Y si borras algo por error, `git reflog` guarda todo lo que hizo HEAD: ahí puedes encontrar el commit "perdido" y volver con `git switch -c recuperada HASH`.

10. .gitignore y archivos que se subieron por error

El archivo `.gitignore` (en la raíz) lista rutas que Git debe ignorar: credenciales (.env), `__pycache__`, `node_modules`, etc. **Solo afecta a archivos NO trackeados todavía**.

Cuando un archivo ya está trackeado y lo quieres sacar del seguimiento (pero conservarlo en tu disco):

```
git rm --cached get-pip.py          # sale del índice; el archivo sigue en tu PC
git rm -r --cached python_mysql/__pycache__
git commit -m "Deja de trackear get-pip.py y __pycache__"
git push
```

--cached = solo el índice de Git. Sin ese flag, `git rm` borra el archivo también de tu disco.

Cuidado: eso limpia el **futuro**, pero el archivo sigue en el **historial** (en cada commit viejo está). Si lo que hay que sacar es una **contraseña**, ver la siguiente sección: hay que reescribir la historia completa.

11. Borrar archivos del historial (credentials o archivos gigantes)

Si subiste por error un archivo con secretos o un archivo enorme, basta con `git rm`: el dato queda en el historial y en GitHub para siempre. Hay que **reescribir la historia**:

Opción A — git filter-repo (la recomendada hoy)

```
pip install git-filter-repo

# 1) Clona una copia limpia (filter-repo exige repo fresco)
git clone --no-local https://github.com/usuario/Agenda-de-contactos.git limpia
cd limpia

# 2) Elimina el archivo de TODA la historia
git filter-repo --path .env --invert-paths
#    ("invierte" el filtro: conserva todo MENOS las rutas indicadas)

# 3) Vuelve a conectar tu remoto (filter-repo borra el "origin")
git remote add origin https://github.com/usuario/Agenda-de-contactos.git

# 4) Sube la historia reescrita (Fuerza: reemplaza la remota)
git push --force origin main

# 5) Repite para cada rama: git push --force origin otra-rama
```

Otras variantes útiles: `--path get-pip.py --invert-paths` para un archivo residual, o `--strip-blobs-bigger-than 10M` para quitar archivos gigantes del historial.

Opción B — git filter-branch (antigua, lenta, en desuso)

```
FILTER_BRANCH_SQUELCH_WARNING=1 git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch .env' --prune-empty --tag-name-filter cat -- --all
```

Hoy se considera obsoleta: **filter-repo** es más rápida, más segura y más fácil.

Obligatorio después de reescribir historial

- **Cambia la contraseña** de MySQL/MySQL y cualquier clave que estuvo en el archivo: borrar del repo NO la borra de los clones, caches de GitHub ni de quien ya la descargó.
- Coordina con tu equipo: todos deben clonar de nuevo o hacer un hard reset al nuevo historial.
- Si el repo es privado y el archivo no era secreto crítico, a veces basta con avisar y rotar credenciales; no siempre vale la pena reescribir historia.

12. GitHub: pull requests, issues y buenas prácticas

- **Pull Request (PR)**: propuesta de fusión de tu rama en otra. Sirve para que otro revise el código, comente y apruebe antes de fusionar. Flujo clásico: rama `feature/x` → PR → revisiones → "Merge pull request" → borrar la rama.
- **Issues**: tareas, bugs y mejoras trackeados en el repo; se pueden vincular a PRs con "Closes #12".
- **Protected branches**: en Settings → Branches puedes prohibir el push directo a main y exigir PR + revisor + checks verdes.
- **Secrets**: Settings → Secrets and variables → Actions: variables cifradas para pipelines; NUNCA pongas contraseñas en el YAML.
- **Tags y releases**: marca versiones con `git tag v1.0.0` → `git push origin v1.0.0` y crea un "Release" con changelog.
- **gh**: la CLI oficial de GitHub (`gh pr create`, `gh issue list`, `gh run watch`).

13. Pipelines en GitHub (GitHub Actions)

Un **pipeline** (integración continua / CI/CD) es una serie de pasos que GitHub ejecuta solo, en sus servidores, cada vez que pasas algo (push, PR, tag, horario). Sirve para ejecutar tests, hacer lint, construir imágenes Docker o desplegar.

Los pipelines viven en archivos YAML dentro de la carpeta `.github/workflows/` (pueden ser varios). Estructura:

```
# .github/workflows/ci.yml
name: CI del proyecto                    # nombre visible en la pestaña Actions

on:                                      # ¿QUÉ dispara el pipeline?
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:                                    # trabajos (corren en paralelo entre sí)
  test:
    runs-on: ubuntu-20.04                # SO del servidor virtual (runner)
    steps:                               # pasos DENTRO del trabajo
      - uses: actions/checkout@v4        # 1) clona tu repo en el runner
      - name: Configurar Python
        uses: actions/setup-python@v5    # 2) instala Python
        with:
          python-version: "3.12"
      - name: Instalar dependencias
        run: pip install -r requirements.txt   # 3) comando de terminal (bash)
      - name: Ejecutar pruebas
        run: python -m unittest discover tests
  build:
    needs: test                          # solo corre si "test" pasó
    runs-on: ubuntu-20.04
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: echo "Compilando el proyecto"
```

Conceptos del YAML

- **on**: disparadores. Más allá de push/PR: `schedule` (cron), `workflow_dispatch` (botón manual).
- **jobs**: trabajos independientes que corren en máquinas limpias distintas; `needs` crea dependencias (build después de test).
- **steps**: cada paso usa o bien `uses:` (una acción reutilizable del marketplace, p. ej. actions/checkout) o bien `run:` (comandos shell).
- **runs-on**: el sistema operativo del runner: ubuntu, windows, macos.

Ejemplo completo: test + build y subir imagen a Docker Hub

```
name: CI y publicar imagen Docker
on:
  push:
    branches: [ main ]
    tags: [ "v*" ]                       # también al etiquetar v1.0.0

jobs:
  test:
    runs-on: ubuntu-20.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python -m unittest discover tests

  docker:
    needs: test
    runs-on: ubuntu-20.04
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3    # inicia sesión en Docker Hub
        with:
          username: ${{ secrets.DOCKERHUB_USER }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: usuario/agenda:latest
```

${{ secrets.X }} se reemplaza en tiempo de ejecución con los valores cifrados de Settings → Secrets. Nunca escribas tokens directamente en el YAML ni los "imprimas" en los logs.

Estado del pipeline: cada paso y cada job muestran un check verde o rojo; en los PRs, un check rojo bloquea la fusión si activaste "Require status checks". Los **artifacts** guardan archivos resultantes (reports, builds) para descargarlos desde la ejecución. El registro completo de cada corrida está en la pestaña **Actions** de GitHub (también con `gh run watch`).

14. Cheatsheet Git/GitHub

```
# Primeros pasos
git init / git clone URL
git add . && git commit -m "mensaje"

# Estado e historial
git status / git log --oneline --graph --all / git diff

# Ramas
git switch -c feature-x / git switch main
git merge feature-x / git rebase main

# Remoto
git remote add origin URL / git push -u origin main
git fetch origin / git pull

# Deshacer
git restore archivo / git restore --staged archivo
git reset --hard HEAD~1 (LOCAL) / git revert HASH (PUBLICADA)
git stash push / git stash pop

# Rescatar / investigar
git reflog / git blame archivo / git bisect start

# Sacar archivo trackeado (sin borrarlo de tu disco)
git rm --cached archivo

# Reescribir historia para borrar un archivo con secretos
git filter-repo --path .env --invert-paths
git push --force origin main
=> y cambiar la contraseña inmediatamente
```

PARTE II: Docker

15. Qué es Docker y para qué sirve

Docker empaca tu aplicación **junto con todo lo que necesita para correr** (versión de Python, librerías, variables, sistema base) dentro de una **imagen**, y esa imagen se ejecuta igual en tu PC, en el servidor de tu jefe o en la nube. Se acabó el clásico *"en mi máquina sí funciona"*.

- **Contenedor**: la instancia en ejecución de una imagen (proceso aislado).
- **Imagen**: la plantilla solo-lectura (tipo "snapshot" instalado).
- **Dockerfile**: receta paso a paso para construir la imagen.
- **Registry**: almacén de imágenes (Docker Hub es el público por defecto; también GHCR de GitHub, ECR de AWS...).

Contenedor vs Máquina Virtual

```
|                     | Contenedor              | Máquina Virtual           |
|---------------------|-------------------------|---------------------------|
| Qué incluye         | tu app + libs + un trozo | SO COMPLETO (GBs)         |
|                     | mínimo del SO (MBs)     |                           |
| Arranque            | segundos                | minutos                   |
| Peso                | ligero (10-200 MB)      | pesado (1-4 GB)           |
| Aislamiento         | comparte el kernel del  | hardware virtualizado     |
|                     | host (cgroups/namespaces)| completo                  |
```

16. Instalación y comandos básicos

Instala **Docker Desktop** (Windows/Mac) o el motor de Docker en Linux. Verifica con:

```
docker --version                    # versión del motor
docker run hello-world              # prueba: descarga una imagen y la corre
```

El ciclo de vida más común

```
docker build -t agenda:1.0 .        # 1) construye la imagen desde el Dockerfile
                                        #    -t nombre:etiqueta   . = carpeta con el contexto
docker images                       # 2) lista las imágenes locales
docker run -d --name agenda -p 8000:8000 agenda:1.0
                                        # 3) crea y arranca un contenedor
                                        #    -d  = detached (a segundo plano)
                                        #    -p  host:contenedor (expone el puerto)
docker ps                           # 4) contenedores en ejecución
docker ps -a                        #    incluidos los detenidos
docker logs agenda                  # 5) ver la salida (stdout) del contenedor
docker logs -f agenda               #    en vivo (como tail -f)
docker exec -it agenda bash         # 6) entrar DENTRO del contenedor a inspeccionar
docker stop agenda                  # 7) detener (SIGTERM, espera)
docker kill agenda                  #    detener a la fuerza (SIGKILL)
docker rm agenda                    # 8) borra el contenedor detenido
docker rmi agenda:1.0               #    borra una IMAGEN
docker system prune -a              # limpieza general: contenedores/imagenes/cache muertos
```

17. El Dockerfile

Ejemplo para una app Python (la de tu agenda, adaptada):

```
# Dockerfile
FROM python:3.12-slim               # imagen base: Python 3.12 minimalista
WORKDIR /app                        # carpeta de trabajo dentro del contenedor
COPY requirements.txt .             # copia solo requirements.txt primero
RUN pip install --no-cache-dir -r requirements.txt   # instala dependencias
COPY python_mysql/ ./python_mysql/  # copia el código del proyecto
EXPOSE 8000                         # documenta que el contenedor usa el puerto 8000
CMD ["python", "python_mysql/python_mysql.py"]   # comando al arrancar
```

Para construir y probar:

```
docker build -t agenda:1.0 .
docker run --rm -it agenda:1.0
```

Instrucciones clave

- **FROM**: siempre la primera línea. De ahí sale la base. Usa variantes ligeras: `python:3.12-slim`, alpine, etc.
- **COPY** vs **ADD**: COPY solo copia archivos; ADD además baja URLs y descomprime tar. Para la mayoría de casos: COPY.
- **RUN**: ejecuta un comando **durante la construcción** (instalar paquetes).
- **CMD** vs **ENTRYPOINT**: ambos definen el comando al **arrancar**. CMD es sobrescribible por defecto (`docker run imagen otro_comando`); ENTRYPOINT fija el ejecutable y CMD solo los argumentos.
- **ENV**: define variables de entorno dentro de la imagen.
- **ARG**: como ENV pero solo existe al construir (no queda en la imagen final).
- **VOLUME** y **EXPOSE**: más que generar cambios, **documentan** que la app usa volumen o puerto (la exposición real se hace con `-p` al correr).

18. Optimización: capas, caché y .dockerignore

Cada instrucción del Dockerfile crea una **capa** cacheada. Si cambias una línea, Docker reconstruye ESE capa y todas las de abajo. Por eso el orden importa:

```
# MAL: copias TODO y luego instalas; al cambiar 1 línea de código, se reinstala todo
COPY . .
RUN pip install -r requirements.txt

# BIEN: dependencias primero (caché válida mientras no cambie requirements.txt)
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY python_mysql/ ./python_mysql/
```

Crea también un **.dockerignore** en la carpeta del contexto (igualito al .gitignore pero para Docker): así no envías el código y la caché no se invalida por archivos inútiles.

```
# .dockerignore
.git
__pycache__
*.pyc
.env
docs
venv
```

> **Los secretos en .env NO deben viajar dentro de la imagen.** Las imágenes se comparten fácilmente y extraer un .env copiado con COPY es trivial. Pásalos en tiempo de ejecución con `docker run --env-file .env` o `-e DB_PASSWORD=...`

Multi-stage builds para reducir tamaño: una primera etapa compila con todas las herramientas y una segunda copia solo los artefactos a una imagen limpia:

```
# etapa 1: compilar
FROM python:3.12 AS builder
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt --target=/app/lib

# etapa 2: ejecutar, solo con lo necesario
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /app/lib /app/lib
COPY python_mysql/ ./python_mysql/
CMD ["python", "python_mysql/python_mysql.py"]
```

19. Volúmenes: persistir datos fuera del contenedor

El sistema de archivos del contenedor es **efímero**: si lo borras con `docker rm`, sus datos desaparecen. Para datos que deben sobrevivir (bases de datos, uploads) usamos **volumes**:

```
docker volume create datos-mysql      # crea un volumen con nombre
docker run -d --name mysql \
  -e MYSQL_ROOT_PASSWORD=erik123 \
  -v datos-mysql:/var/lib/mysql \
  -p 3306:3306 mysql:8                # la BD vive en el VOLUME, no en el contenedor

docker volume ls                      # lista volúmenes
docker volume inspect datos-mysql     # ¿dónde vive físicamente?
# si borras el contenedor, los datos siguen en el volumen
```

Bind mount: en vez de un volumen gestionado por Docker, enlazas una carpeta de tu PC. Ideal en desarrollo para que los cambios se vean al instante:

```
docker run -it -v "C:/proyecto/python_mysql:/app/python_mysql" agenda:1.0
# Windows: usa la ruta completa con / o \; en PowerShell escapa con doble barra
```

- **Volumen (-v nombre:destino)**: lo administra Docker; portable entre sistemas.
- **Bind mount (-v ruta-host:destino)**: enlaza tu carpeta real; rápido para desarrollo, mala idea para producción.

20. Redes

Por defecto todos los contenedores comparten la red `bridge` y se ven por nombre:

```
docker network create agenda-net
docker run -d --name mi-mysql --network agenda-net mysql:8
docker run -d --name mi-api --network agenda-net agenda:1.0
# dentro de mi-api, "mi-mysql" resuelve como host. SIN tocar IPs ni puertos.
```

Puertos: `-p 8080:80` = "del PC, puerto 8080 va al puerto 80 del contenedor". Regla: `-p puerto_de_tu_PC:puerto_del_contenedor`.

21. docker-compose: varios servicios con un solo archivo

Para orquestar la app completa (por ejemplo tu agenda + MySQL) sin teclear 20 comandos de `docker run`, defines todo en un `docker-compose.yml`:

```
# docker-compose.yml
services:                             # (antes se llamaba "version:" + "services")
  mysql:
    image: mysql:8                    # usa una imagen ya publicada
    container_name: mi-mysql
    restart: unless-stopped           # se reinicia si se cae o si reinicias la PC
    environment:                      # variables (como el .env pero aquí)
      MYSQL_ROOT_PASSWORD: erik123
      MYSQL_DATABASE: clientesdb
    ports:
      - "3306:3306"
    volumes:
      - datos-mysql:/var/lib/mysql    # los datos persisten en el volumen
    healthcheck:                      # cómo saber si el servicio está sano
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  agenda:
    build:                            # construye desde TU Dockerfile
      context: .
      dockerfile: Dockerfile
    container_name: mi-agenda
    depends_on:                       # arranca después de que mysql esté sano
      mysql:
```

```
        condition: service_healthy
    environment:
      DB_HOST: mysql                  # el nombre del servicio hace de host
      DB_PASSWORD: erik123
    ports:
      - "8000:8000"

volumes:                              # declara el volumen que usaste
  datos-mysql:

docker compose up -d                   # levanta todo en segundo plano (construye si hace falta)
docker compose ps                     # estado de los servicios
docker compose logs -f agenda         # logs de UN servicio
docker compose down                   # para y borra contenedores y red (NO borra volúmenes)
docker compose down -v                # también borra los volúmenes (¡cuidado: borra datos!)
docker compose up --build             # reconstruye antes de levantar
```

depends_on con healthcheck resuelve el clásico problema de arranque: la app arranca antes de que MySQL acepte conexiones y truena. `condition: service_healthy` espera a que el check pase.

22. Variables de entorno con Docker

Tres formas de pasar configuración a un contenedor:

```
# 1) En el comando (mala para secretos: queda en el historial del shell)
docker run -e DB_USER=root -e DB_PASSWORD=erik123 agenda:1.0

# 2) Con --env-file (lee un .env; NO viaja dentro de la imagen)
docker run --env-file .env agenda:1.0

# 3) En docker-compose.yml, sección environment: o env_file: agenda:
#    environment:
#      DB_HOST: mysql
#    env_file:
#      - .env
```

23. Buenas prácticas de Docker

- No corras procesos como root dentro del contenedor: crea un usuario con `RUN useradd` y `USER`.
- Etiqueta tus imágenes por versión (`agenda:1.0`), no solo `latest`: te permite volver a versiones conocidas.
- Los secretos van por variables de entorno en tiempo de ejecución, nunca con `COPY .env .` dentro de la imagen.
- Aprovecha el orden de capas para que la caché funcione (dependencias antes que código).
- Define `restart: unless-stopped` en servicios de producción.
- Usa `healthcheck` en servicios que otros dependen (bases de datos, cachés).
- Cierra puertos que no necesites y no uses `--network host` salvo que lo entiendas: anula el aislamiento.
- Limpia tu PC de vez en cuando: `docker system prune -a`.

24. Docker + GitHub Actions (imágenes en el pipeline)

Ya viste en la Parte I un job que construye y publica. Flujo típico: push a main → Actions construye la imagen con `docker build` → la sube a Docker Hub o GHCR con `docker push` (o las acciones oficiales `docker/login-action`, `docker/build-push-action`) → el servidor hace `docker pull` y reinicia el contenedor. Así tu app se despliega sola sin copiar archivos a mano.

25. Cheatsheet Docker

```
docker build -t nombre:1.0 .          # construir imagen
docker images / docker ps -a
docker run -d --name c1 -p 8080:80 -v vol:/datos --env-file .env imagen:1.0
docker logs -f c1 / docker exec -it c1 bash
docker stop c1 / docker rm c1 / docker rmi imagen:1.0
docker volume ls / docker network ls
docker system prune -a               # limpieza general

docker compose up -d / docker compose down / docker compose logs -f
docker compose down -v                # ¡borra también los datos!
docker compose up --build            # reconstruir y levantar
```

Referencias para seguir aprendiendo: documentación oficial de Git (git-scm.com/book), GitHub Docs (docs.github.com), documentación de Docker y Docker Hub. Práctica recomendada: usa Git todos los días en este proyecto de la agenda y levanta la agenda con docker-compose cuando añadas la BD.